

Actor Components

Base class for all components of the `UActorComponent`. Parts are the only way to provide meshes and images, execute collisions, and play audio, so that when a player is playing or interacting with a game, a piece of code will eventually work.

There are a few main classes to understand when creating your own parts: the actor part, the visual part and the primitive part.

- Actor components or Class `UActorComponent` can be used for abstract behaviour such as movement, inventory or feature management and some other non-physical concepts. Actor parts do not really change, which also means that they have no physical location or rotation in the world.
- The View component (the child of the class `USConnect` component, `UActorComponent`) supports location-based behaviours that do not require geometric representation. This includes spring weapons, cameras, physical forces and obstacles (but not physical objects) and audio.
- Primitive components or child of class `UPrimitiveComponent`, `USceneComponent` are the visual components with their geometric representations and are usually used to provide visual elements or to overlap or overlap with physical objects. This includes static or skeletal meshes, sprite or billboards and particle systems as well as boxes, capsules and circular sections.

Registering Components

To update each frame of the actor components and affect the view, the engine must record them. It automatically occurs to the parts created as a by-product of the actor during that actor's expansion process. However, manual registration for parts created during the game is available. The register component function provides this functionality, this component is associated with an actor.

Unregistering Components

To remove an actor's parts from the upgrade, simulation, or rendering processes, you cannot register it with an undocumented component function.

Updating

The actor's parts are capable of updating each frame, just like the cast. The `TickComponent` function allows components to execute code in each frame.